

FGV EMap
João Pedro Jerônimo

Aprendizado Profundo

Revisão para A1

Rio de Janeiro
2026

Conteúdo

1	Segmentação Semântica	4
1.1	Introdução e Métricas	5
1.1.1	Métricas de Avaliação	5
1.1.2	Evolução das Abordagens	6
1.2	Ferramentas Fundamentais	8
1.2.1	Técnicas de Upsampling	8
1.2.1.1	Max Unpooling	8
1.2.1.2	Transpose Convolution	9
1.2.2	Mecanismos de Contexto e Campo Receptivo	10
1.2.2.1	Atrous Convolution	11
1.2.2.2	Image Pooling (Global Average Pooling)	11
1.2.3	Convoluções Eficientes	12
1.2.3.1	Atrous Spatial Pyramid Pooling (ASPP)	12
1.2.3.2	Convolução Normal V.S Separable Convolution & Atrous Separable Convolution	13
1.2.4	Blocos Residuais	15
1.3	Arquiteturas	15
1.3.1	SegNet	15
1.3.2	U-Net	16
1.3.3	ResUNet	16
1.3.4	DeepLab V1 & V2	18
1.3.5	PARSENet	19
1.3.6	PSPNet	19
1.3.7	Deeplab V3 & V3+	20
1.4	Percas	21
1.4.1	Cross Entropy Loss	21
1.4.2	Balanced Cross Entropy Loss	22
1.4.3	Focal Loss	22
1.4.4	Balanced Focal Loss	22
1.4.5	Loss Function for Regression	22
1.5	Pontos Práticos	23
1.5.1	A necessidade de Patches (Tiles)	23
1.5.2	O Problema do Contexto nas Bordas (Border Context Loss)	23
1.5.3	CNN Patch-wise Clássica vs. FCN Modela	24
2	Object Detection	25
2.1	Introdução	26
2.2	Métricas de Avaliação	26
2.3	Redes de Estágio Único (Single-Shot): A Família YOLO	27
2.3.1	Fundamentos do YOLO	27
2.3.2	Parametrização do vetor de saída	27
2.3.3	Caixas de Ancoragem (Anchor Boxes)	27
2.3.4	Supressão Não-Máxima (Non-Maximum Suppression - NMS)	28
2.3.5	Evolução Arquitetural	29
2.3.5.1	YOLOv1 & YOLO9000	29
2.3.5.2	YOLOv3	30
2.3.6	Loss Function	30

2.4	Redes de Dois Estágios e Segmentação de Instâncias: Mask R-CNN	31
3	Recurrent Neural Networks (RNNs)	32
4	Generative Adversarial Networks (GANs)	34

Segmentação Semântica

1.1 Introdução e Métricas

A segmentação semântica é uma tarefa de visão computacional que consiste em classificar cada pixel de uma imagem em uma categoria específica. Essa tarefa é fundamental para diversas aplicações, como direção autônoma, análise médica e realidade aumentada.

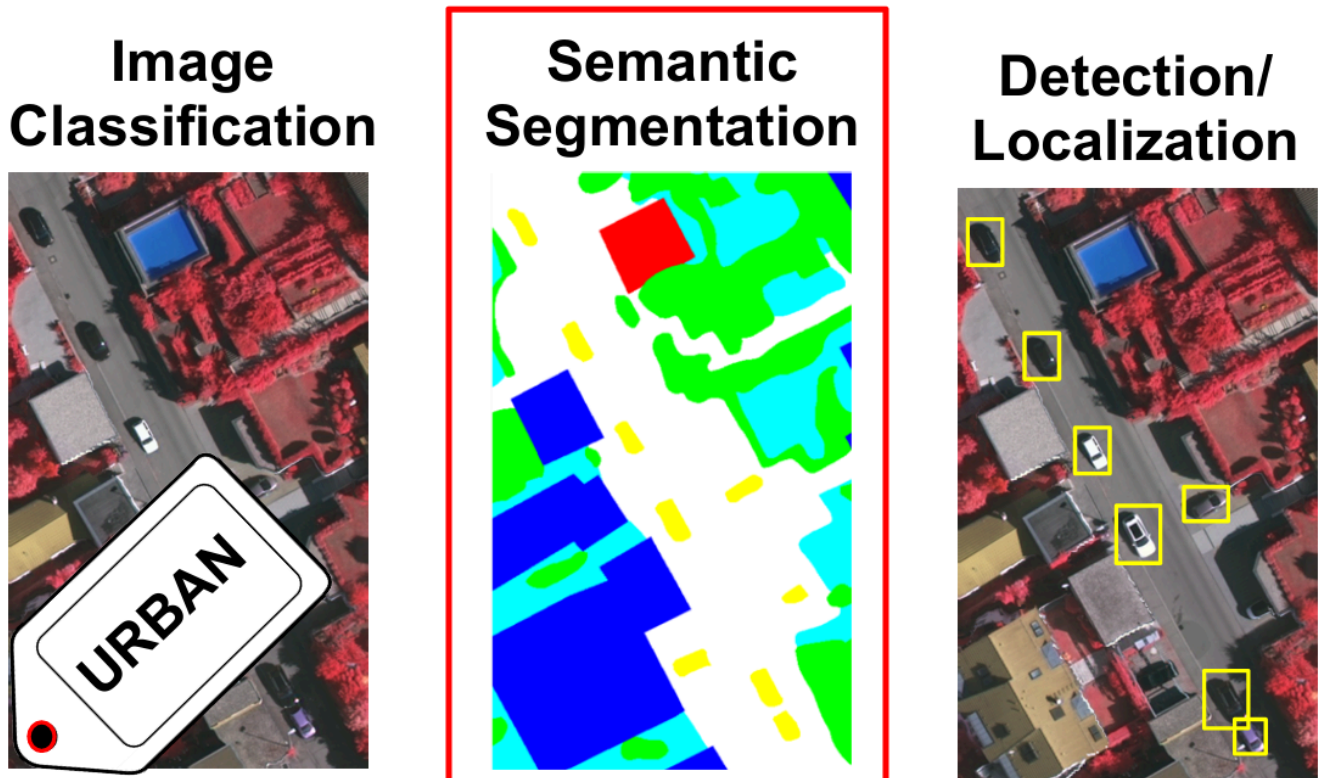


Figura 1: Áreas de visão computacional

Dentro do contexto de segmentação, vale diferenciar entre segmentação semântica e segmentação de instâncias. A segmentação semântica atribui uma classe a cada pixel, enquanto a segmentação de instâncias não apenas classifica os pixels, mas também distingue entre diferentes objetos da mesma classe.

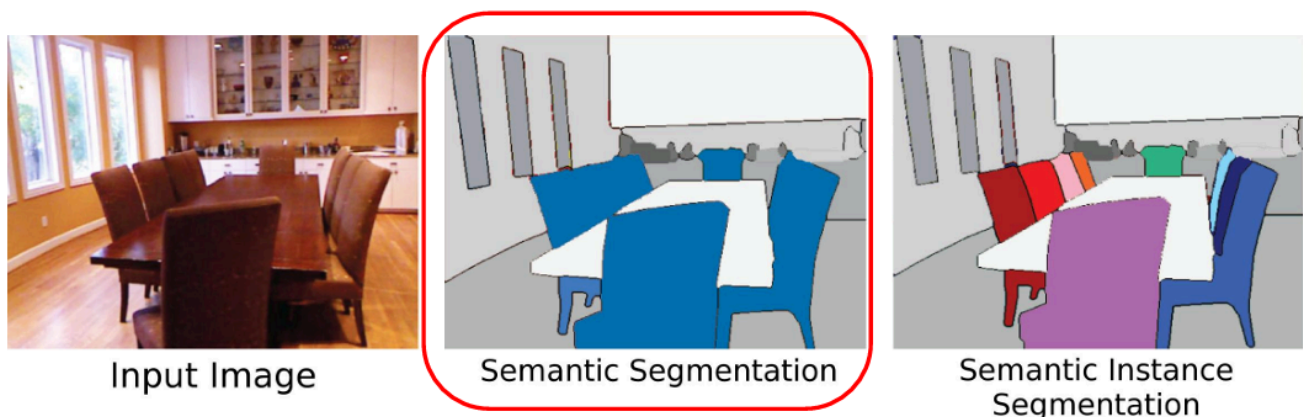


Figura 2: Diferença entre segmentação semântica e segmentação de instâncias

1.1.1 Métricas de Avaliação

Podemos utilizar algumas métricas para avaliar a performance de um modelo de segmentação semântica. As métricas mais comuns incluem:

Definição 1.1.1.1 (Intersection over Union (IoU)):

$$\text{IoU} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}} \quad (1)$$

Definição 1.1.1.2 (Precision):

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2)$$

Definição 1.1.1.3 (Average Precision): Dado que na minha imagem eu tenho mapeado K classes, a métrica de Average Precision (AP) é definida como a média das precisões de cada classe:

$$\text{AP} = \left(\frac{1}{K} \right) * \sum_{k=1}^K \text{Precision}_k \quad (3)$$

1.1.2 Evolução das Abordagens

Vamos relembrar como é estruturada uma rede convolucional padrão para classificação de uma imagem.

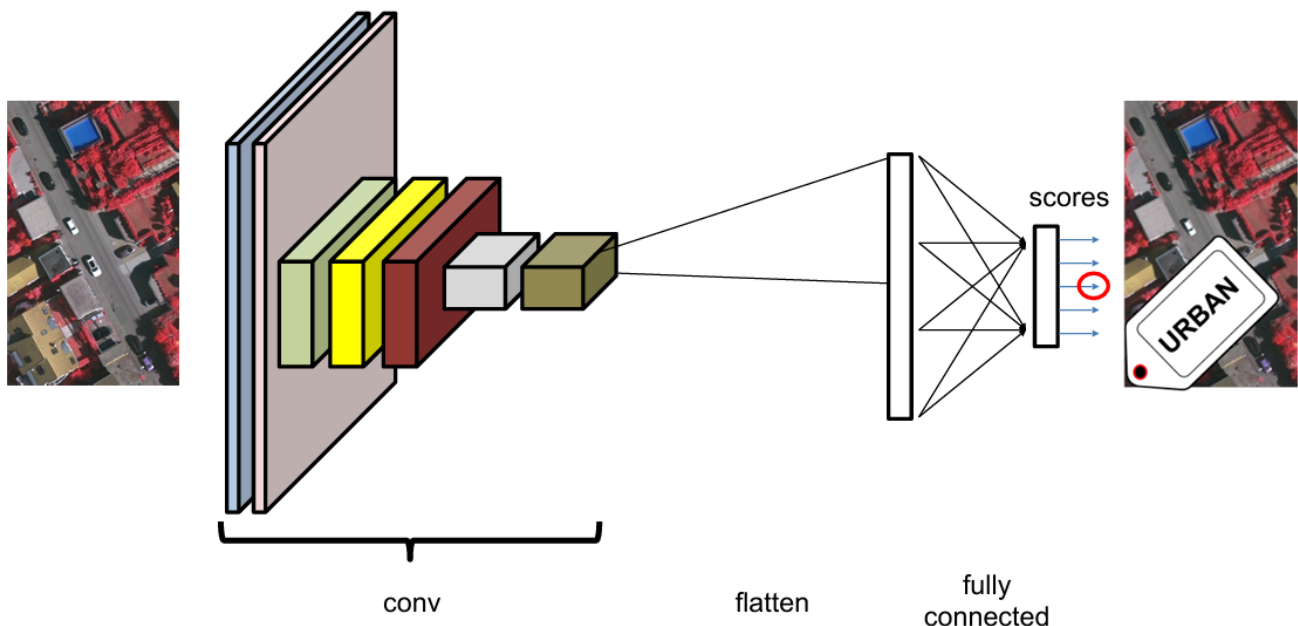


Figura 3: Estrutura de uma rede convolucional padrão

A imagem passa por uma série de canais de convolução, pooling e normalização, que extraem características relevantes. No final, temos uma camada totalmente conectada que produz a classificação final. Isso nos faz pensar em uma ideia, que tal termos uma janela que percorre a imagem e classifica cada pixel individualmente? Essa abordagem é conhecida como “sliding window” e é uma das primeiras tentativas de segmentação semântica.

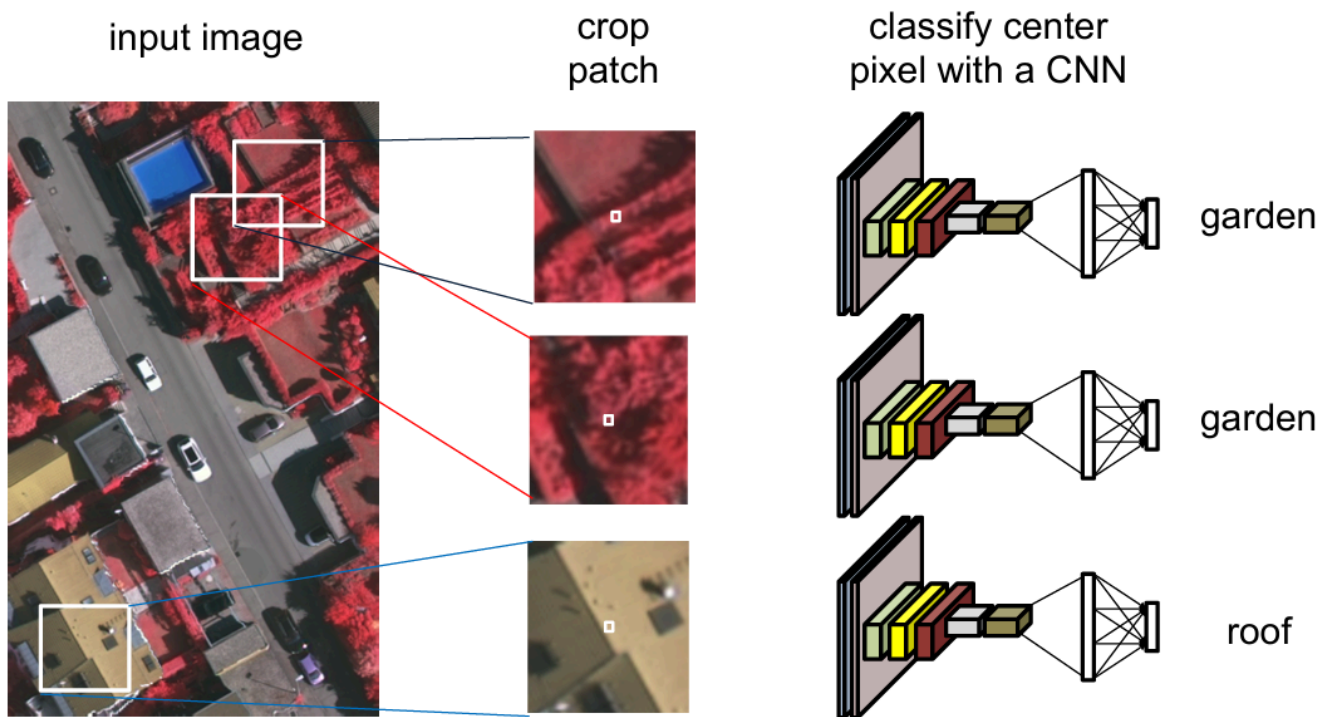


Figura 4: Abordagem de sliding window

No entanto, essa abordagem é computacionalmente cara e não aproveita o contexto global da imagem. Para superar essas limitações, surgiram as Fully Convolutional Networks (FCNs), que substituem as camadas totalmente conectadas por camadas convolucionais, permitindo que a rede produza mapas de segmentação diretamente.



Figura 5: Estrutura de uma Fully Convolutional Network (FCN)

Essa abordagem é interessante, já que permite que a rede aprenda a segmentar a imagem de forma mais eficiente, utilizando o contexto global e local. Além disso, as FCNs podem ser treinadas de forma end-to-end, o que simplifica o processo de treinamento. No entanto, as FCNs são MUITO pesadas, e por isso surgiu a ideia do **downsampling** e **upsampling** dentro da rede, onde basicamente diminuimos os tamanhos das features internas e depois vamos reconstruindo até o tamanho original da imagem. Essa abordagem é conhecida como **encoder-decoder**.

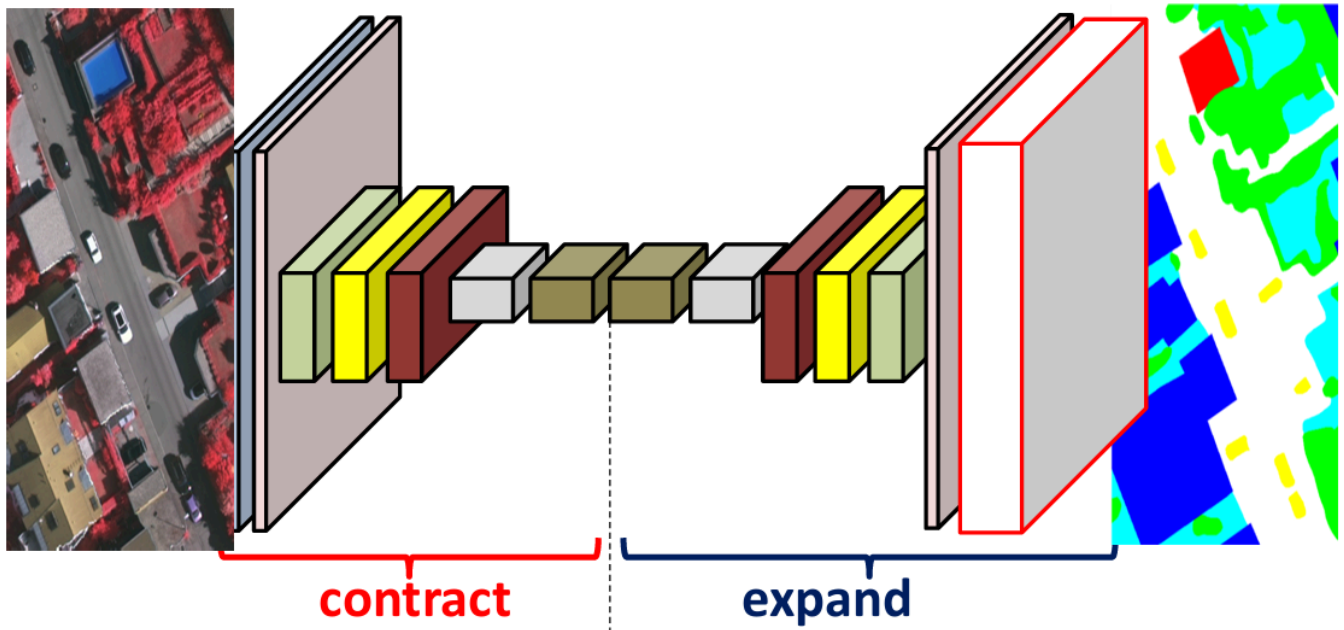


Figura 6: Estrutura de uma rede encoder-decoder

1.2 Ferramentas Fundamentais

Antes de irmos para as arquiteturas, precisamos relembrar algumas ferramentas fundamentais que são utilizadas em redes de segmentação semântica, vamos definir todas as ferramentas que iremos utilizar ao apresentar nossas arquiteturas, mas vale ressaltar que existem ferramentas que **caracterizam** algumas arquiteturas, como o **skip connection** na U-Net e os blocos residuais na ResUNet, mas isso não significa que essas ferramentas não podem ser utilizadas em outras arquiteturas, por exemplo, podemos utilizar blocos residuais na SegNet, mas isso não caracteriza a arquitetura como uma ResUNet.

1.2.1 Técnicas de Upsampling

O processo de upsampling é o processo de reconstrução da dimensão original das features internas da rede, que foram reduzidas pelo **downsampling**. Existem diversas técnicas para realizar o upsampling, como o **max unpooling** e a **transpose convolution**, que serão detalhadas a seguir.

1.2.1.1 Max Unpooling

Relembrando o que é o **pooling**, ele é uma operação que reduz a dimensionalidade das features internas da rede, geralmente utilizando operações como **max pooling** ou **average pooling**. O **unpooling** é o processo inverso, onde tentamos reconstruir a dimensão original das features a partir das features reduzidas. No entanto, o **unpooling** não é uma operação trivial, pois não temos informações suficientes para reconstruir a dimensão original de forma precisa.

O que acontece é que, **antes** de fazer o **pooling**, nós guardamos os índices dos valores máximos (no caso do **max pooling**), e depois utilizamos esses índices para reconstruir a dimensão original durante o **unpooling**. Essa abordagem é conhecida como **max unpooling**.

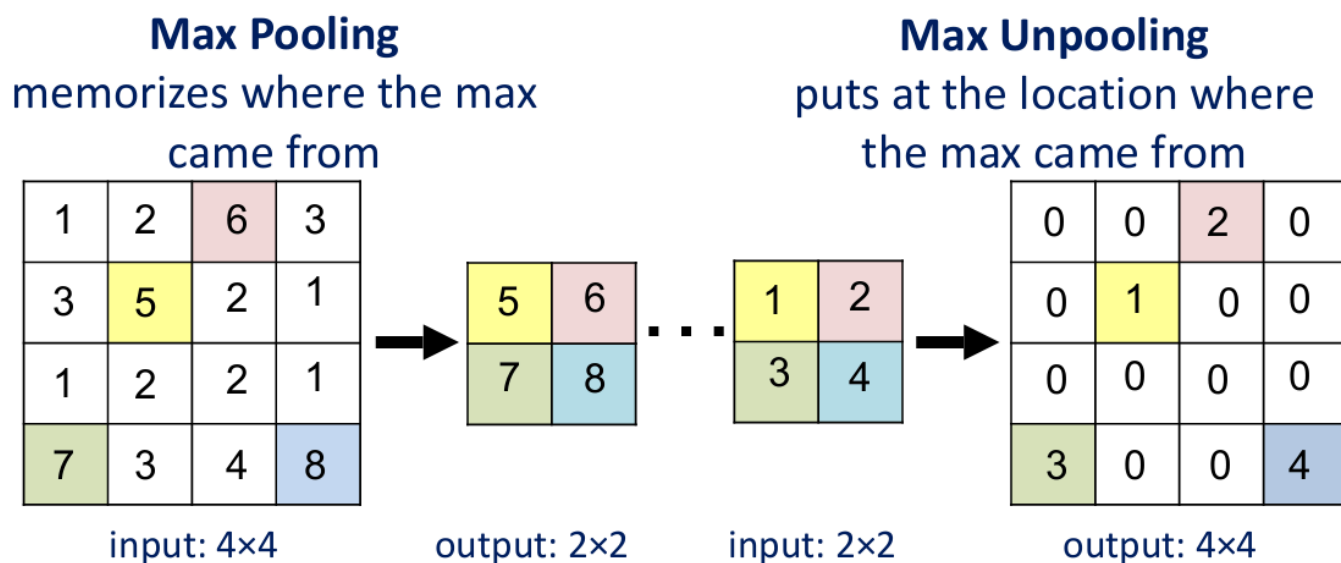


Figura 7: Exemplo de max unpooling

1.2.1.2 Transpose Convolution

A desvantagem do **max unpooling** é que ele depende dos índices dos valores máximos, o que pode limitar a capacidade da rede de aprender representações mais complexas. Uma alternativa é utilizar a **transpose convolution**, também conhecida como **deconvolution**. Essa operação é semelhante à convolução, mas ao invés de reduzir a dimensionalidade das features, ela aumenta.

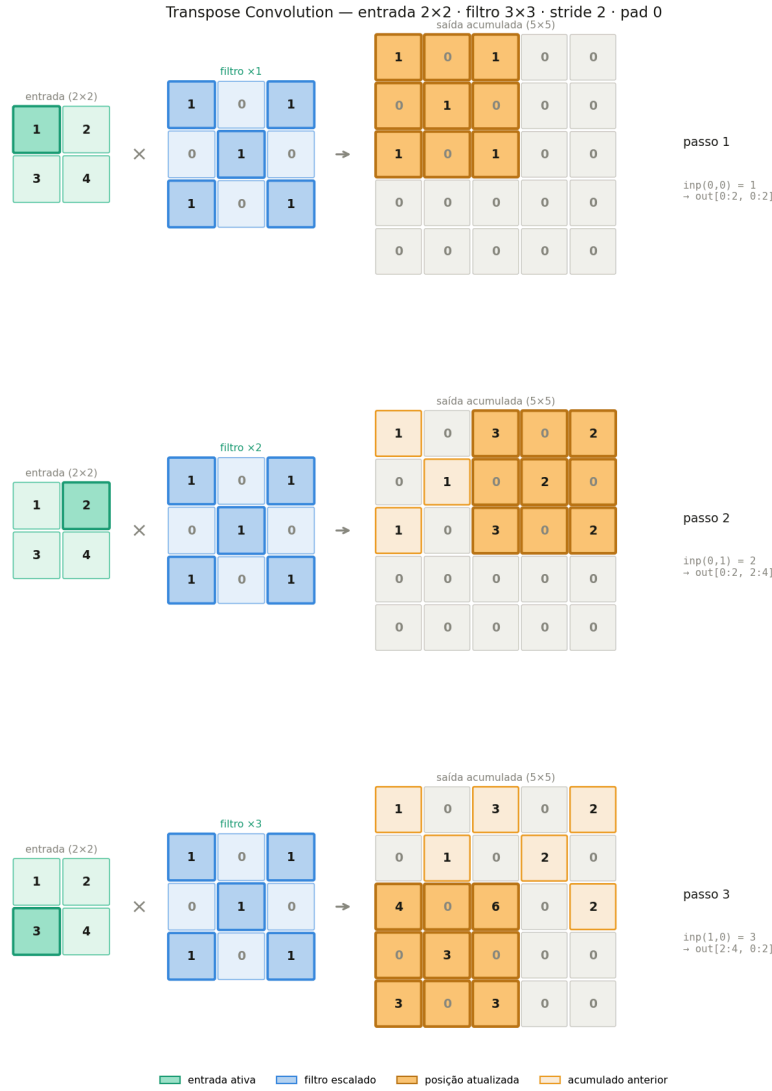


Figura 8: Exemplo de transpose convolution

Como vimos na disciplina de **machine learning**, podemos obter uma matriz de filtro chamada K_{col} a partir da operação *im2col*, que transforma a imagem em uma matriz de colunas. A **transpose convolution** é basicamente a operação inversa, onde aplicamos a matriz de filtro K_{col} na matriz obtida anteriormente

Se X é a entrada da convolução e K é o filtro tal que

$$Y = X * K \quad (4)$$

Já sabemos que

$$Y = K_{\text{col}} X_{\text{col}} \quad (5)$$

temos então que a transpose convolution é definida como

$$X_{\text{col}} = K_{\text{col}}^T Y \quad (6)$$

de tal forma que os X_{col} são reconstruídos a partir dos Y e do filtro K_{col} (não de forma perfeita pois há perda de informação na compressão do X para o Y , mas o objetivo é que a rede aprenda a reconstruir o X da melhor forma possível)

1.2.2 Mecanismos de Contexto e Campo Receptivo

Antes de irmos de fato para as arquiteturas, temos que definir um conceito usado em algumas delas, se não vamos interromper o raciocínio no meio do caminho. O conceito é o de **contexto**

global, que basicamente é a ideia de que, para classificar um pixel, precisamos levar em consideração não apenas os pixels vizinhos, mas também os pixels mais distantes da imagem. Só que usar filtros maiores consome mais memória, tempo de processamento e não permite que a rede aprenda a extrair características mais complexas.

1.2.2.1 Atrous Convolution

Para resolver esse problema, surgiu a ideia de **atrous convolution**, que é uma técnica que permite aumentar o tamanho do filtro sem aumentar o número de parâmetros da rede nem redimensioná-la. A ideia é inserir “buracos” (ou **holes**) entre os pixels do filtro, permitindo que ele “veja” mais pixels da imagem sem aumentar o número de parâmetros.

Definição 1.2.2.1.1 (Atrous Convolution): Dado um filtro K de tamanho $k \times k$, e uma feature map X , a **atrous convolution** é definida como

$$Y(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X(i + r * m, j + r * n) K(m, n) \quad (7)$$

onde r é o **rate** de atrous convolution, que determina o espaçamento entre os pixels do filtro.

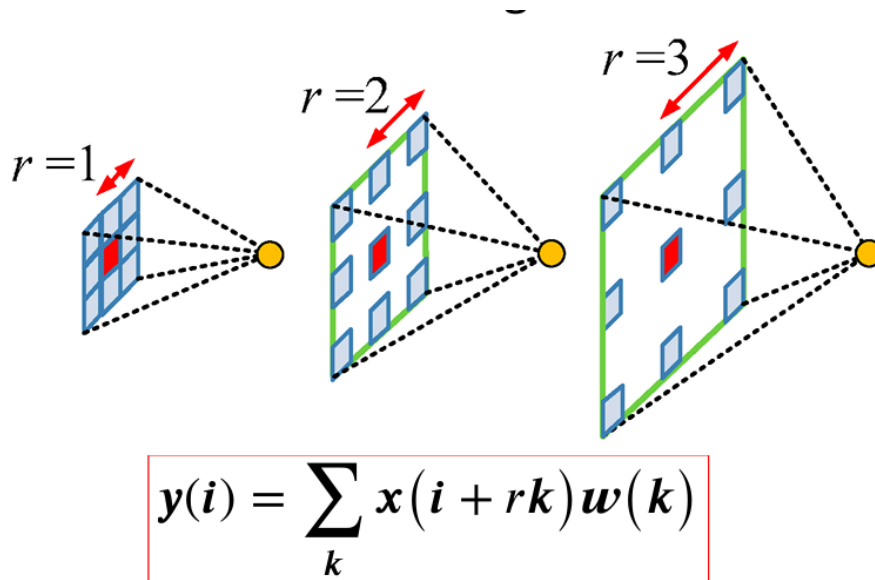


Figura 9: Exemplo de atrous convolution

1.2.2.2 Image Pooling (Global Average Pooling)

Essa é uma abordagem que permite trazer contexto global da imagem para a rede. Pegamos um feature map e aplicamos um average pooling em toda a imagem, obtendo um vetor de características que representa a imagem como um todo. Esse vetor é então redimensionado através de upsampling e concatenado com o feature map original, permitindo que a rede utilize informações de contexto global para melhorar a segmentação.

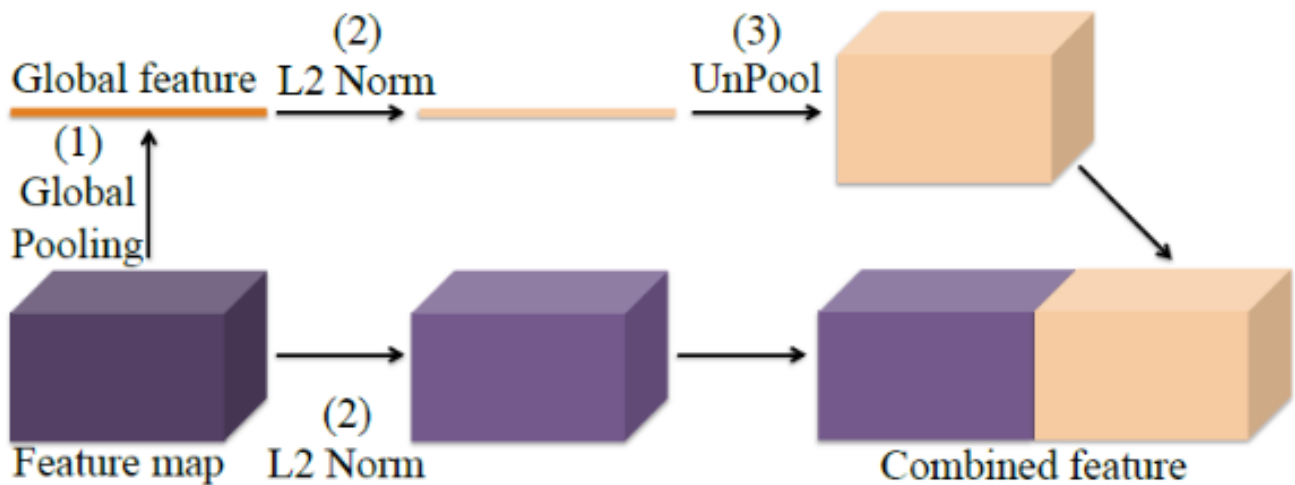


Figura 10: Exemplo de image pooling

1.2.3 Convoluções Eficientes

Como já discutimos n vezes, as convoluções são operações matemáticas bem caras, e por isso estamos sequer discutindo essas arquiteturas, pois elas são pensadas não somente para a melhor classificação da rede, mas para que possam ser eficientes!

1.2.3.1 Atrous Spatial Pyramid Pooling (ASPP)

Esse conceito é bem simples, vimos anteriormente sobre as Atrous Convolution, convoluções espaçadas para obtenção de contexto local da imagem. No entanto, esse passo pode ser tanto benéfico quanto maléfico! Imagine que a feature que queremos identificar é muito grande no conjunto das imagens, se a rate for muito pequena, isso pode prejudicar que os filtros consigam capturar essa feature, e o mesmo vale para features muito pequenas, se a rate for muito grande, isso pode prejudicar que os filtros a visualizem. A ideia do ASPP é justamente utilizar múltiplas rates de atrous convolution, para que possamos capturar features de diferentes tamanhos na imagem, de forma que elas rodem **paralelamente** e depois sejam concatenadas, permitindo que a rede utilize informações de diferentes escalas para melhorar a segmentação.

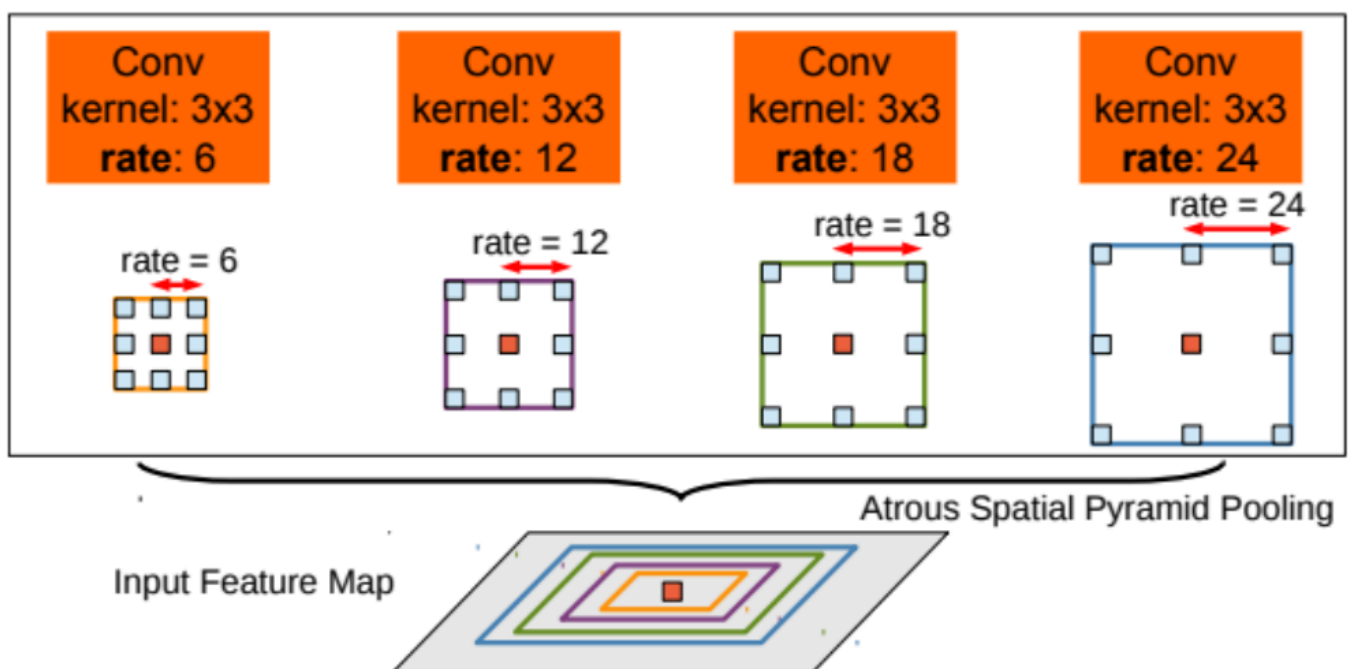


Figura 11: Exemplo de atrous spatial pyramid pooling

1.2.3.2 Convolução Normal V.S Separable Convolution & Atrous Separable Convolution

Em uma convolução normal, cada filtro é aplicado a todos os canais da imagem, o que resulta em um grande número de operações. Veja a imagem a seguir, por exemplo

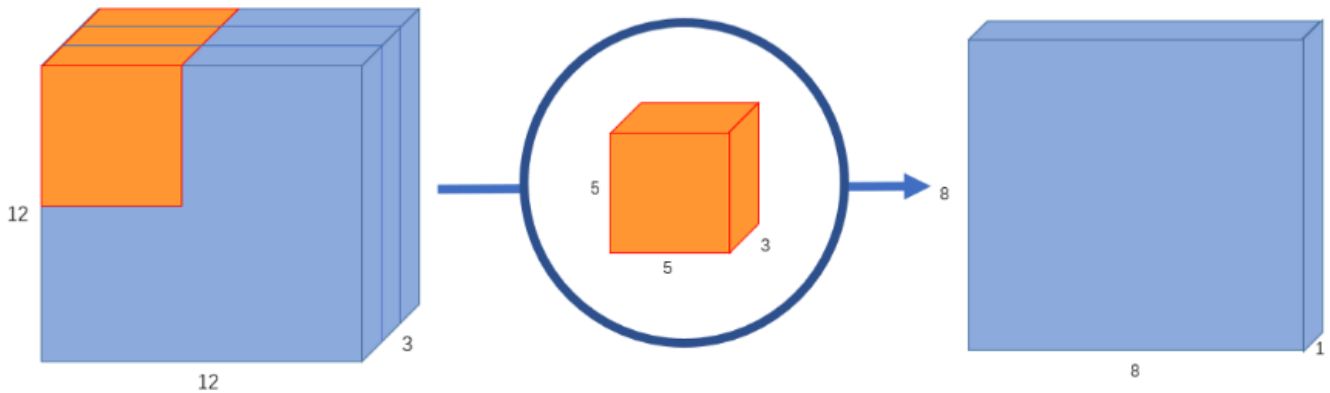


Figura 12: Exemplo de imagem com 3 canais

perceba que, ao aplicar o filtro na imagem com 3 canais, toda a informação foi sumariada em um único canal (assim, se quiséssemos várias feature maps, nós aplicaríamos C filtros diferentes para obter C feature maps). Como nosso filtro é $5 \times 5 \times 3$, temos um total de 75 parâmetros. Como a imagem é de tamanho 12×12 , o filtro vai percorrer a imagem 8×8 vezes, resultando em um total de $75 * 64 = 4800$ operações. Agora imagine o cenário que falei de utilizarmos múltiplos filtros, vamos usar por exemplo $C = 256$

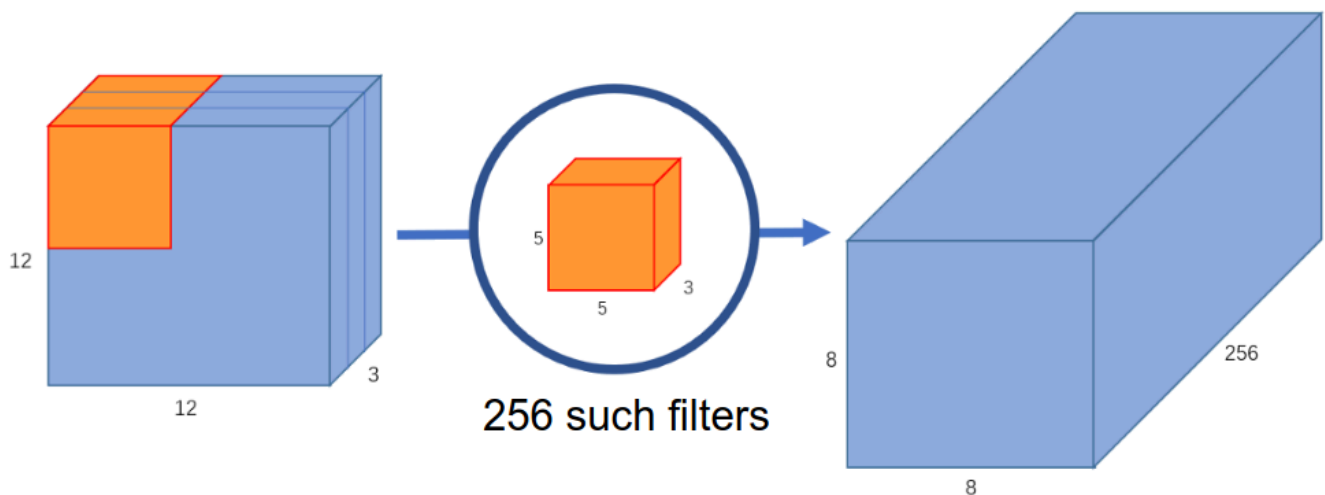


Figura 13: Exemplo de imagem com 3 canais e 256 filtros

Nesse exemplo, temos 256 filtros de tamanho $5 \times 5 \times 3$, resultando em um total de $75 * 256 = 19200$ parâmetros. Como a imagem é de tamanho 12×12 , o filtro vai percorrer a imagem 8×8 vezes, resultando em um total de $19200 \cdot 64 = 1.228.800$ operações. Isso é muito caro computacionalmente, e por isso surgiram as convoluções separáveis.

Agora vamos entender como funciona o processo da **SEPARABLE CONVOLUTION**, ela é dividida em 2 etapas, a **depthwise convolution** e a **pointwise convolution**. Na **depthwise convolution**, cada filtro é aplicado a apenas um canal da imagem, resultando em múltiplos feature maps, um para cada canal

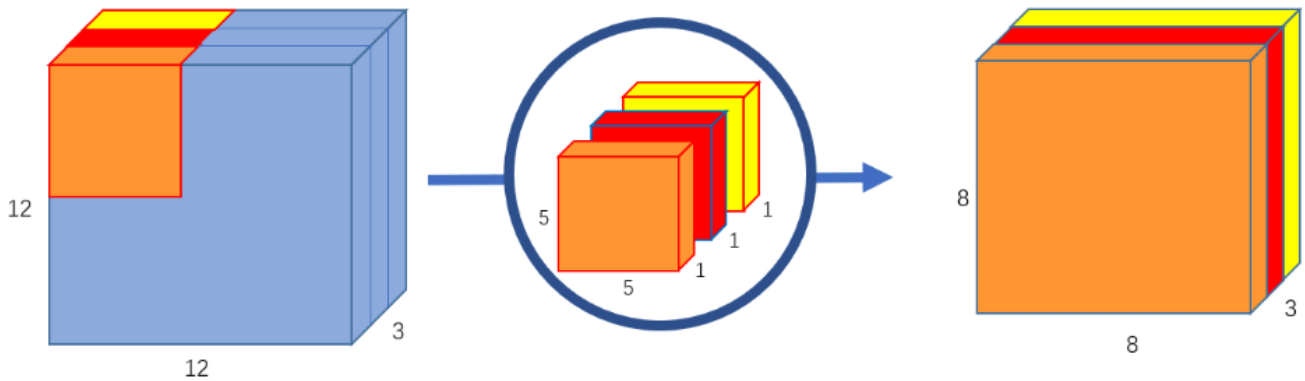


Figura 14: Exemplo de depthwise convolution com 3 canais

ainda existem os mesmos 75 parâmetros, já que são 3 filtros de tamanho 5×5 , mas agora temos 3 feature maps, um para cada canal. Agora, na **pointwise convolution**, aplicamos um filtro 1×1 em cada feature map, resultando em múltiplos feature maps, um para cada filtro (no mesmo estilo que o filtro antigo era aplicado na convolução normal)

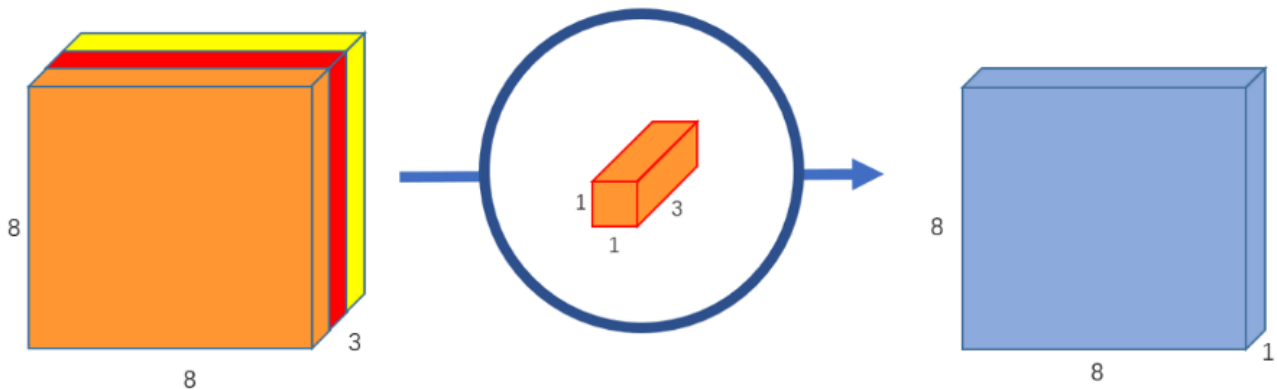


Figura 15: Exemplo de pointwise convolution com 3 canais

Agora temos que nosso número de parâmetros, além dos 75 do filtro anterior, temos mais esse filtro menor, nos resultando em 78 parâmetros. O número de multiplicações, na primeira etapa, é $75 \times 8 \times 8 = 4.800$, adicionando com a segunda etapa, temos $4800 + (1 \times 1 \times 3) \times 8 \times 8 = 4.800 + 192 = 4992$. No entanto, para obtermos múltiplos feature maps, vamos utilizar $C = 256$ filtros dos 1×1 que comentamos, dessa forma:

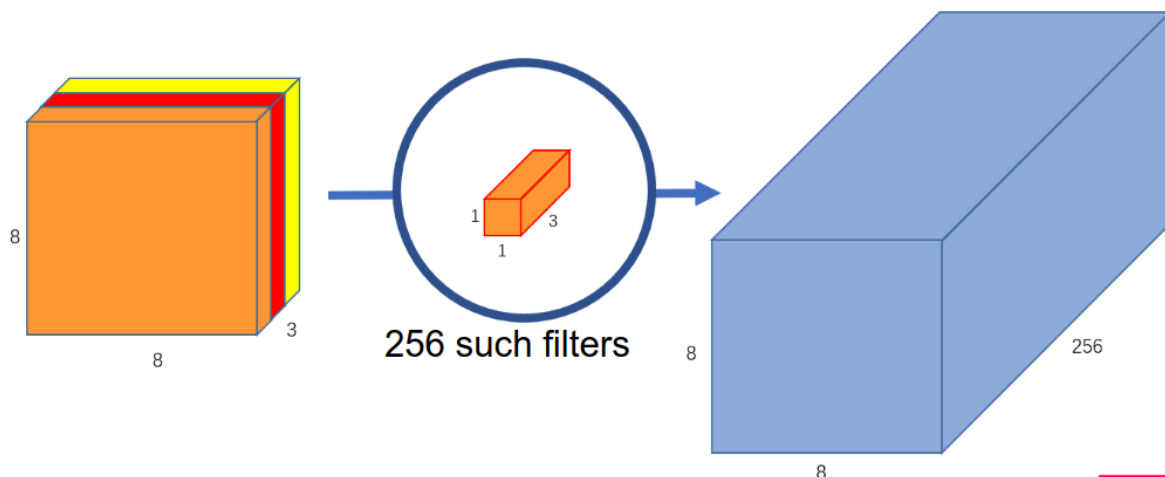


Figura 16: Exemplo de pointwise convolution com 3 canais e 256 filtros

Agora vamos ter que a quantidade de filtros é o filtro inicial mais os 256 outros filtros, logo: $75 + (1 \times 1 \times 3) \times 256 = 75 + 768 = 843$ parâmetros. E como aumentamos apenas os filtros 1×1 para obter os nossos 256 feature maps, o número de multiplicações passa a ser $4800 + (1 \times 1 \times 3) \times (8 \times 8) \times 256 = 4800 + 49152 = 53952$.

Esse conceito pode ser expandido para as convoluções Atrous, de forma que as mudanças necessárias são mínimas, mantendo tracking do rate r conseguimos aplicar o mesmo conceito de **depthwise** e **pointwise** para as convoluções Atrous, resultando em uma redução significativa no número de parâmetros e operações, mantendo a capacidade da rede de capturar informações de diferentes escalas.

1.2.4 Blocos Residuais

Normalmente, redes neurais são straight-to-the-point, nós temos a entrada x e a partir disso a rede modela uma função complexa F tal que

$$y = F(x) \quad (8)$$

no entanto, pode existir casos em que y é MUITO parecido com x com leves ajustes, e isso, surpreendentemente, pode dificultar muito o aprendizado da rede. Para consertar isso, os chamados **blocos residuais** foram introduzidos, de forma que a rede não aprende a relação direta entre x e y , mas sim a **diferença** entre eles (o quão diferente y é de x), ou seja, a rede aprende uma função F tal que

$$y = F(x) + x \quad (9)$$

O principal motivo dessa abordagem é o **gradiente no backpropagation**. Em redes comuns de deep-learning, o gradiente pode se tornar muito pequeno (ou até mesmo zero) à medida que é propagado para trás, dificultando o aprendizado. Com os blocos residuais, o gradiente pode fluir diretamente através da conexão de atalho, permitindo que a rede aprenda mais facilmente.

1.3 Arquiteturas

1.3.1 SegNet

A SegNet é uma arquitetura de rede neural convolucional projetada para segmentação semântica. Ela segue a arquitetura padrão que já demonstramos utilizando do método de max unpooling para realizar upscaling

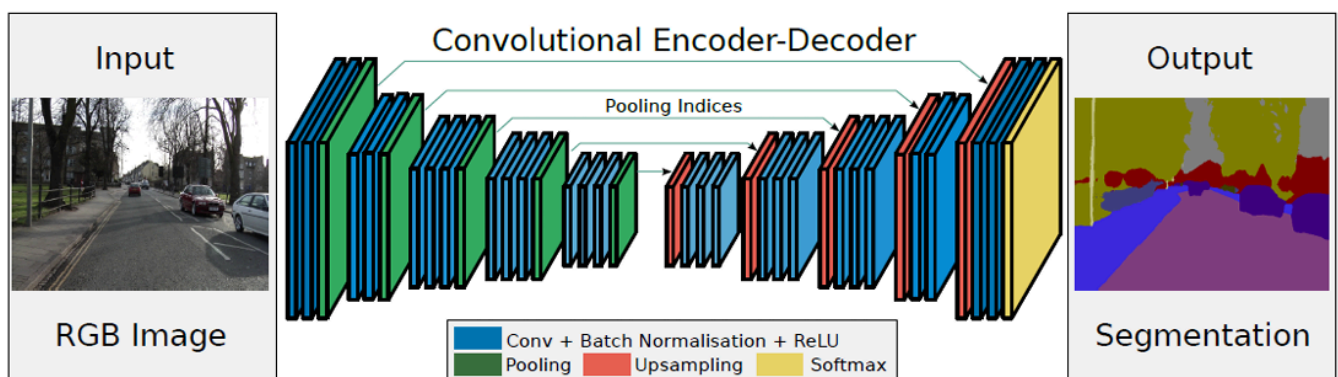


Figura 17: Arquitetura da SegNet

1.3.2 U-Net

Já na U-Net, a arquitetura é um pouco diferente, ela utiliza **skip connections** para conectar as camadas de downsampling com as camadas de upsampling, permitindo que a rede utilize informações de diferentes níveis de abstração para melhorar a segmentação.

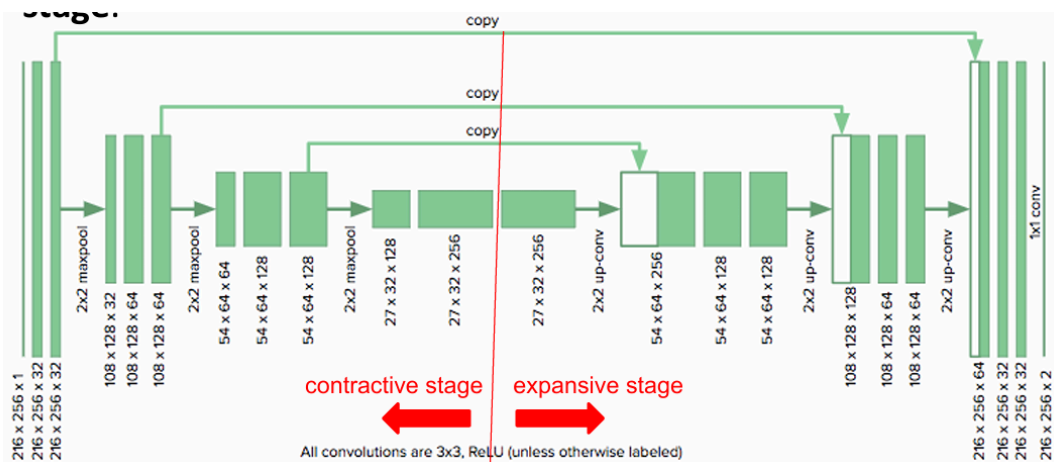


Figura 18: Arquitetura da U-Net

Nas camadas de upsampling, a U-Net utiliza **transpose convolution** para aumentar a dimensionalidade das features unida com um **aumento** nos canais das features. Após o transpose convolution, a U-Net concatena as features da camada correspondente de downsampling, permitindo que a rede utilize informações de diferentes níveis de abstração para melhorar a segmentação, como se ela falasse: “depois de reconstruir a imagem, eu obtive o seguinte mapa de feature, mas lá atrás antes de eu ter feito o downsampling, eu tinha obtido o seguinte mapa de feature, então vou juntar os dois para melhorar a segmentação” (por exemplo, se eu tenho uma imagem 32x32 na escala de cinza, com apenas um canal de cor, na hora do último upsampling, a camada logo após a transpose convolution terá 2 canais “de cor”, que seria o mapa obtido pela rede anteriormente e o mapa obtido na camada de upsampling).

1.3.3 ResUNet

Na ResUNet, a arquitetura é uma combinação da U-Net com blocos residuais, permitindo que a rede aprenda a diferença entre as features de downsampling e upsampling, melhorando ainda mais a segmentação.



Figura 19: Arquitetura da ResUNet

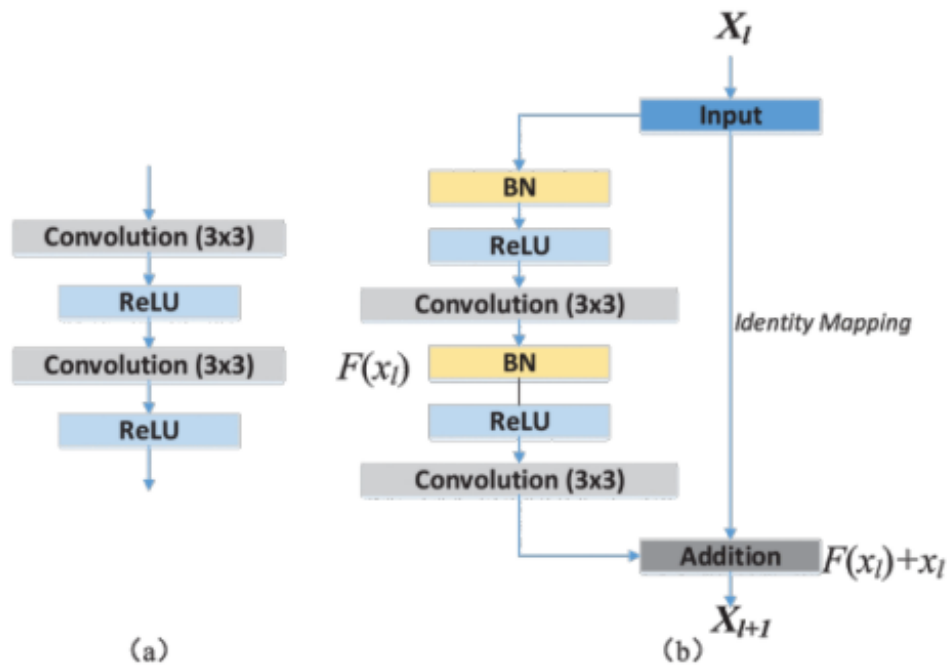


Figura 20: (a) Bloco padrão da UNet. (b) Bloco residual da ResUNet

1.3.4 DeepLab V1 & V2

A DeepLabV1 é uma arquitetura de rede neural convolucional projetada para segmentação semântica, que utiliza **atrous convolution** para aumentar o campo receptivo da rede sem aumentar o número de parâmetros. Ela também utiliza **image pooling** para trazer contexto global da imagem para a rede.

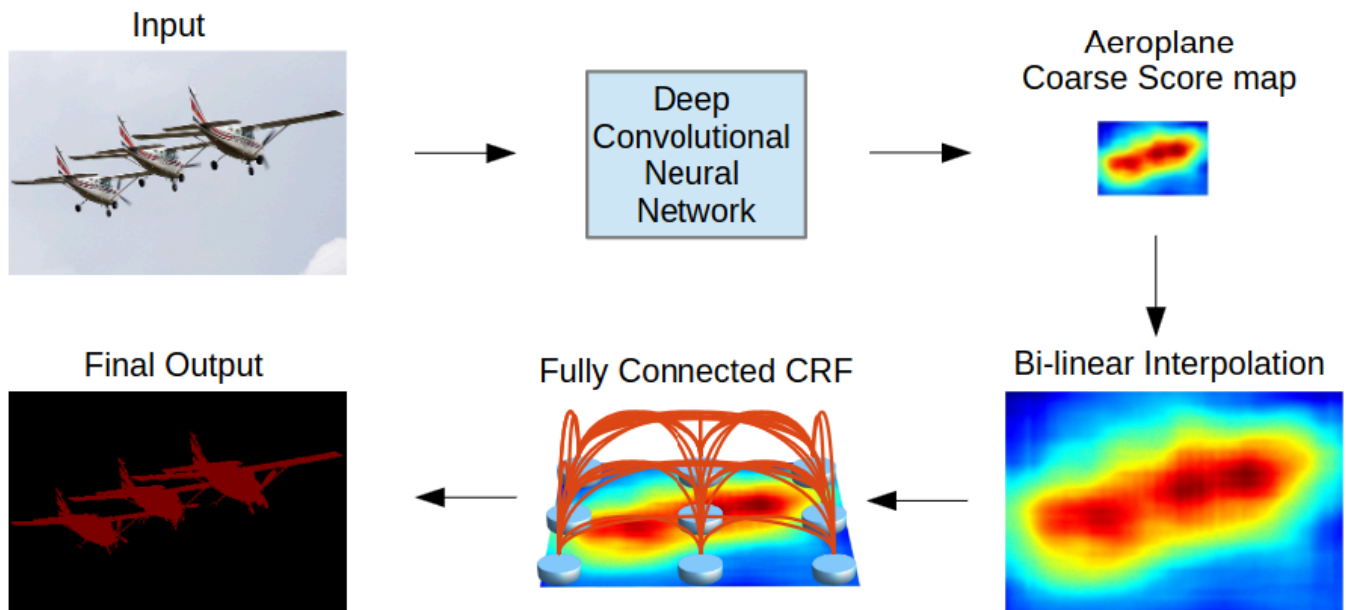


Figura 21: Arquitetura da DeepLabV1&V2

Ambas seguem uma arquitetura muito semelhante, diferindo por um único conceito. Na DeepLab V1, passamos a imagem por uma Deep Convolutional Neural Network (DCNN) para extrair features utilizando camadas de Atrous Convolution. Depois, pegamos o score map obtido e aplicamos um processo de **interpolação bilinear** para aumentar a dimensionalidade do score map, e por fim aplicamos o um algoritmo de pós-processamento chamado **Conditional Random Field (CRF)** para refinar a segmentação. Já na DeepLab V2, o processo é o mesmo, mas ao invés de aplicarmos

apenas uma Atrous Convolution, aplicamos múltiplas Atrous Convolutions com diferentes **rates** (ASPP).

1.3.5 PARSENet

Foi na ParseNet que surgiu a ideia do **image pooling**, gerando o contexto global da imagem para a rede, permitindo que ela utilize informações de diferentes níveis de abstração para melhorar a segmentação. Já vimos antes como esse conceito funciona, mas como ele é estruturado dentro da rede?

Primeiro a rede passa por uma rede convolucional padrão, depois o feature map gerado é passado por um **image pooling**, que gera um vetor de características que representa a imagem como um todo. Esse vetor é então redimensionado através de upsampling e concatenado com o feature map original, permitindo que a rede utilize informações de contexto global para melhorar a segmentação.

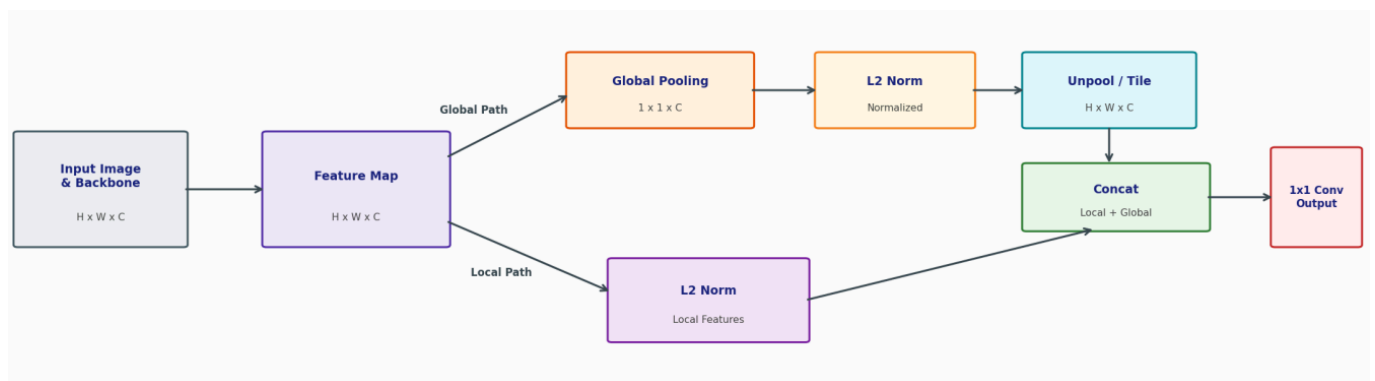


Figura 22: Arquitetura simplificada da PARSENet

1.3.6 PSPNet

A PSPNet é uma arquitetura de rede neural convolucional projetada para segmentação semântica, que utiliza **pyramid pooling** para capturar informações de diferentes escalas da imagem. Ela também utiliza **atrous convolution** para aumentar o campo receptivo da rede sem aumentar o número de parâmetros.

Primeiro a imagem passa por um processamento em uma CNN, essa rede diminui a imagem original para 1/8 do seu tamanho original e entra no bloco PSP **único**. Assim como no ASPP a imagem passa por múltiplas convoluções paralelas, no bloco PSP, a imagem passa por múltiplos **average pooling** com diferentes tamanhos de agrupamento:

- **Nível Red (1 × 1):** Realiza um Global Average Pooling sobre toda a extensão espacial. Gera um único vetor por canal que representa o contexto macro/global da cena inteira
- **Nível Orange (2 × 2):** Divide o mapa em 4 quadrantes (grade 2 × 2) e extrai a média de cada região (contexto regional amplo)
- **Nível Blue (3 × 3):** Divide o mapa em 9 sub-regiões (grade 3 × 3) para capturar um contexto regional médio
- **Nível Green (6 × 6):** Divide o mapa em 36 sub-regiões (grade 6 × 6) para capturar detalhes locais e regionais finos

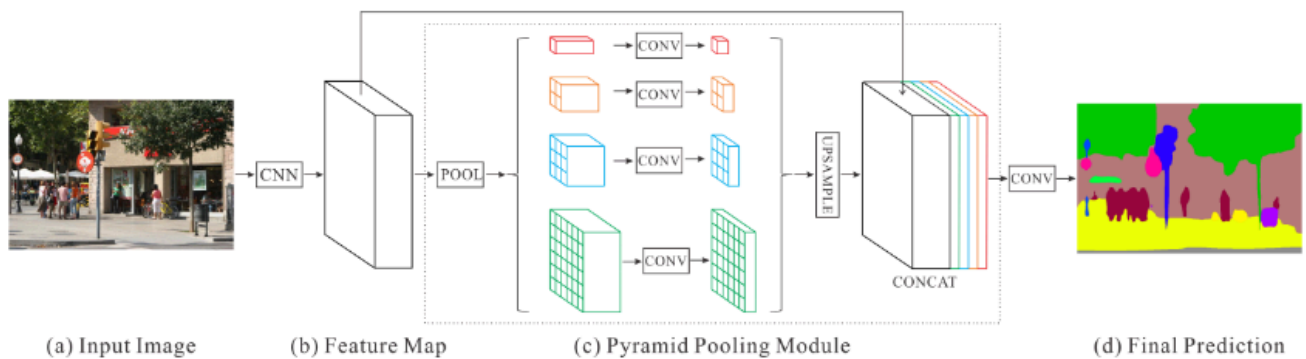


Figura 23: Arquitetura da PSPNet

então através do upsample, cada mapa de pooling é redimensionado para o tamanho original do feature map, e todos os mapas são concatenados, permitindo que a rede utilize informações de diferentes escalas para melhorar a segmentação.

1.3.7 Deeplab V3 & V3+

A DeeplabV3 teve algumas melhorias implementadas. O primeiro ponto foi a remoção do pós-processamento com CRF, que foi substituído por um **upsampling** simples, dessa forma a própria rede consegue aprender a mapear corretamente a segmentação. O segundo ponto foi a implementação do ASPP, que já havíamos comentado anteriormente. E o terceiro ponto foi a implementação, em paralelo com o ASPP, de um **image pooling** para pegar contexto global da rede

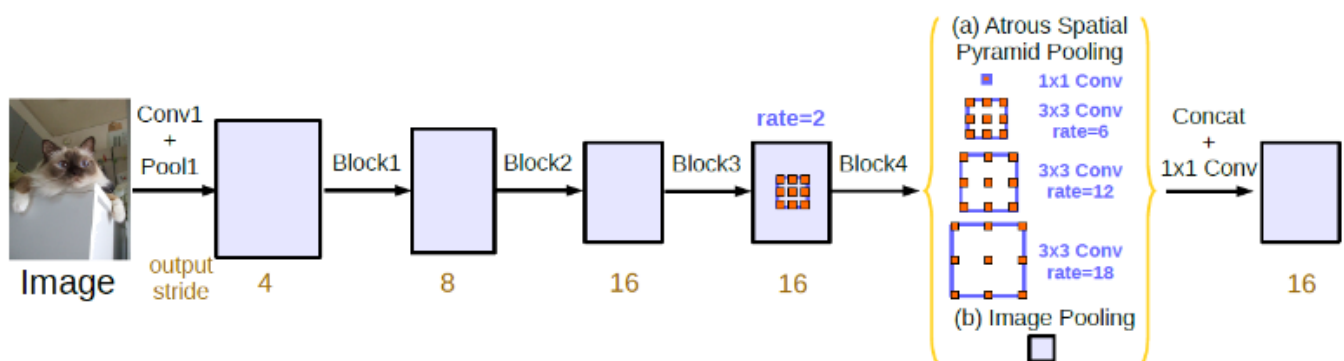


Figura 24: Arquitetura da DeeplabV3

O **DeepLabv3+** foi projetado para resolver uma limitação fundamental do DeepLabv3: embora o DeepLabv3 capturasse um contexto multi-escala excelente através do ASPP, ele perdia detalhes finos e precisão nas bordas dos objetos devido à redução de resolução espacial (**striding** e **pooling**) no backbone. Para corrigir isso, o DeepLabv3+ combina o melhor de duas abordagens: a extração de contexto do **Spatial Pyramid Pooling (ASPP)** com a capacidade de recuperação de bordas da estrutura **Encoder-Decoder**.

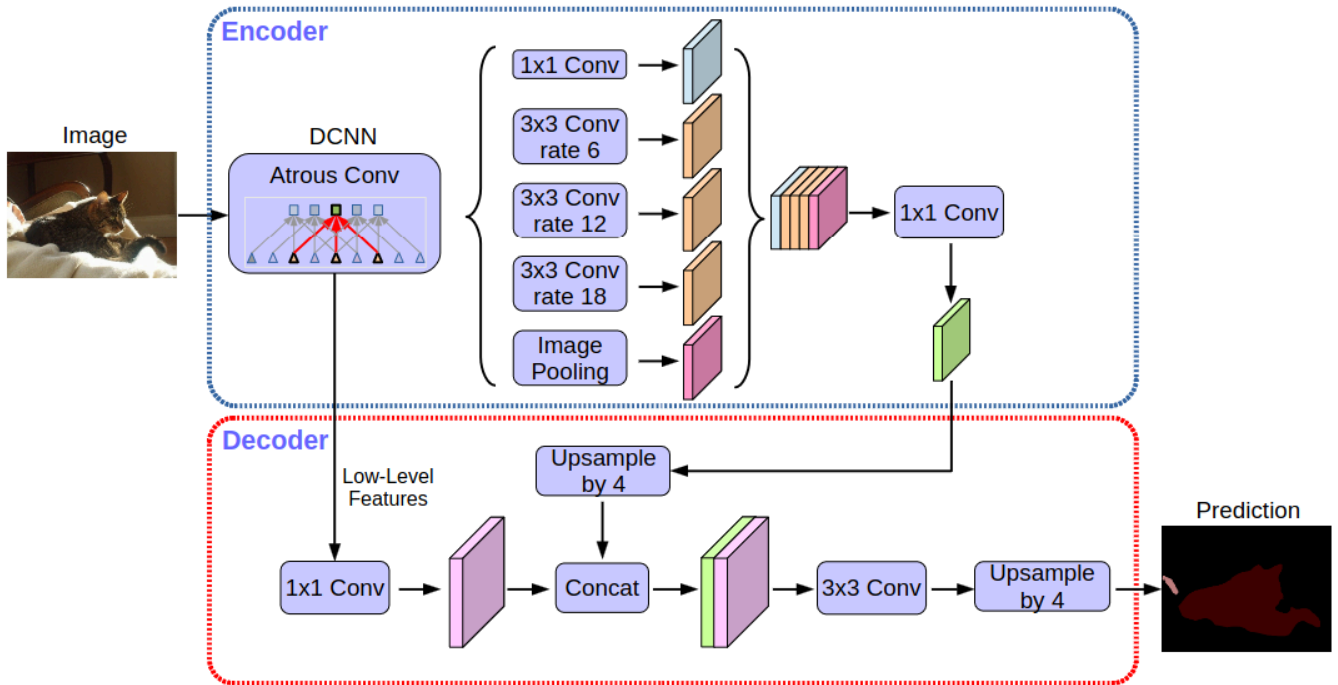


Figura 25: Arquitetura da DeepLabV3+

Em vez do upsampling bilinear direto que o V3 fazia, o V3+ agora tem um módulo decoder dedicado, onde ele faz upsampling das características e vai utilizando das features do encoder para refinar a segmentação, especialmente nas bordas dos objetos. Isso permite que o modelo mantenha a precisão espacial enquanto ainda aproveita o contexto global capturado pelo ASPP.

1.4 Percas

Agora vamos visualizar as losses que utilizamos para informar o modelo como aprender! Mas antes, vamos recapitular alguns pontos e definições.

Primeiramente, dado uma imagem $H \times W$, a saída dos nossos modelos deve ser uma matriz $H \times W$ onde cada elemento da matriz representa a classe do pixel correspondente na imagem (também pode ser um vetor com $N = H \cdot W$ elementos). Vamos considerar a abordagem de um único vetor de N elementos com a classe de cada pixel. Definiremos também t_i sendo a **classe verdadeira** do pixel i (ground truth). Definimos também p_i sendo a probabilidade que o modelo atribui para que i seja da sua **classe verdadeira** ($\mathbb{P}(\text{classe}(i) = t_i)$). A partir disso, podemos definir as losses que utilizamos para treinar nossos modelos.

1.4.1 Cross Entropy Loss

A primeira que vamos ver é a mais padrão para problemas de classificação, a **cross entropy loss**. Ela é definida como

$$\text{CE} = -\frac{1}{N} \sum_{i=1}^N \log(p_i) \quad (10)$$

se o modelo prevê alta probabilidade para a classe correta do pixel, o $\log(p_i)$ será próximo de 0, e a loss será pequena. Se o modelo prevê baixa probabilidade para a classe correta do pixel, o $\log(p_i)$ será negativo e a loss será grande. O objetivo do treinamento é minimizar essa loss, ajustando os pesos da rede para que ela preveja corretamente as classes dos pixels.

No entanto, essa loss carrega um problema. Quando existe um desbalanceamento de classes dentro do meu dataset, pode acontecer de a rede aprender a prever apenas a classe majoritária, ignorando as classes minoritárias.

1.4.2 Balanced Cross Entropy Loss

Para resolver o problema citado, podemos utilizar a **balanced cross entropy loss**, que atribui pesos diferentes para cada classe, penalizando mais os erros nas classes minoritárias.

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N \omega_{t_i} \log(p_i) \quad (11)$$

o peso ω_{t_i} é pré-calculado de forma inversamente proporcional à frequência da classe t_i no dataset, de forma que classes minoritárias tenham pesos maiores e classes majoritárias tenham pesos menores. Isso força a rede a prestar mais atenção às classes minoritárias durante o treinamento. Podemos ter uma formulação binária também

$$\text{BCE} = -\frac{1}{N} \left(\sum_{i \in \text{positivos}} \omega_{\text{pos}} \log(p_i) + \sum_{i \in \text{negativos}} \omega_{\text{neg}} \log(p_i) \right) \quad (12)$$

1.4.3 Focal Loss

Essa loss serve para resolver um problema sutil. A loss anterior resolve o problema de desbalanceamento de classes, mas não resolve o problema de **hard examples**, ou seja, exemplos que são difíceis de classificar e quais são esses pixels? São justamente os que estão cada vez mais próximos das bordas do elemento. No entanto, os pixels fáceis costumam ser os mais presentes, e a soma do gradiente de sua contribuição pode atrapalhar no aprendizado dos pixels mais difíceis. A **focal loss** resolve esse problema, diminuindo a contribuição dos pixels fáceis para o gradiente, permitindo que a rede foque nos pixels mais difíceis.

$$\text{FL} = -\frac{1}{N} \sum_{i=1}^N (1 - p_i)^\gamma \log(p_i) \quad (13)$$

- **Comportamento do pixel fácil:** Consideremos $p_i = 0.95$ e $\gamma = 2$, então o fator de peso fica 0.0025, a perda desse pixel é reduzida em 99.75%, fazendo com que ele quase não afete o treinamento
- **Comportamento do pixel difícil:** Consideremos $p_i = 0.2$ e $\gamma = 2$, então o fator de peso fica 0.64, a perda desse pixel é mantida relevante

γ é o parâmetro focal, quanto maior ele é, mais severa é a penalidade aplicada aos pixels fáceis, e quanto menor ele é, mais leve é a penalidade aplicada aos pixels fáceis. O valor padrão de γ é 2, mas ele pode ser ajustado dependendo do problema e do dataset.

1.4.4 Balanced Focal Loss

Combina as duas soluções, ponderando cada pixel de acordo com sua classe e aplicando penalidade em pixels fáceis, de forma que a rede foque nos pixels mais difíceis e nas classes minoritárias.

$$\text{FL}_{\text{bal}} = -\frac{1}{N} \sum_{i=1}^N \omega_{t_i} (1 - p_i)^\gamma \log(p_i) \quad (14)$$

1.4.5 Loss Function for Regression

A saída por pixel pode não necessariamente ser um label de classe, mas um valor numérico. Por exemplo, se a rede estiver tentando estimar a profundidade aplicada àquela foto, então utilizamos as losses L_1 e L_2

$$L_1 = \frac{1}{N} \sum_{i=1}^N |y_i - t_i|$$

$$L_2 = \frac{1}{N} \sum_{i=1}^N (y_i - t_i)^2$$
(15)

onde y_i é o valor predito pelo modelo e t_i é o valor verdadeiro.

1.5 Pontos Práticos

1.5.1 A necessidade de Patches (Tiles)

Em aplicações reais, as imagens originais frequentemente possuem resoluções massivas (ex: 4000×4000 pixels ou mais). Tentar passar uma imagem desse tamanho inteira por uma rede neural de uma só vez estoura o limite de memória VRAM da GPU.

Por isso, na prática, a imagem é fatiada em blocos menores (**patches** ou **tiles**) para serem processados individualmente durante o treinamento e a inferência, montando-se um **mosaico** com os resultados finais de cada bloco.

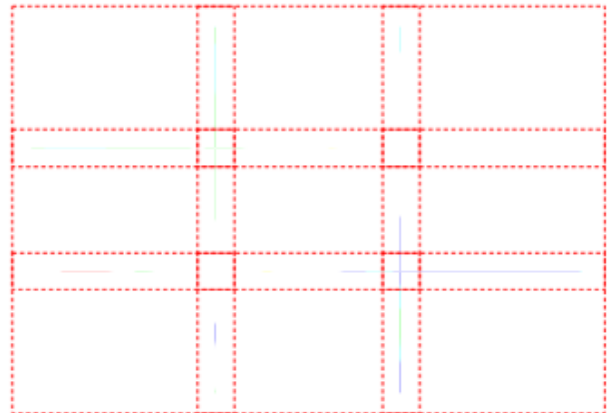


Figura 26: Exemplo de fatiamento de uma imagem em blocos menores

1.5.2 O Problema do Contexto nas Bordas (Border Context Loss)

Dividir a imagem em blocos cria um desafio geométrico nas extremidades de cada bloco:

- **Perda de Contexto Espacial:** Os pixels situados nas bordas de um **patch** perdem a vizinhança e o contexto espacial da região vizinha que ficou no **patch** ao lado.
- **Artefatos no Mosaico:** Ao colar os **patches** de volta para reconstruir a imagem final, essa falta de contexto nas margens gera artefatos de corte visíveis, descontinuidades e erros de classificação ao longo das linhas de junção dos blocos.

Para mitigar a perda de contexto nas bordas dos blocos, podemos destacar a estratégia de *mosaico com sobreposição* (**Overlapping Patches**):

1. **Sobreposição de Blocos:** Os **patches** são recortados com uma porcentagem de sobreposição em relação aos vizinhos, em vez de serem colados estritamente lado a lado.
2. **Considerar Apenas a Região Central (Inner Part):** Descarta-se a borda externa do **patch** (onde o contexto foi prejudicado) e utiliza-se apenas a previsão da região central válida.
3. **Média das Predições (Averaging Results):** Nas áreas onde os blocos se sobrepõem, calcula-se a média das probabilidades previstas por cada bloco para definir a classe final do pixel.

1.5.3 CNN Patch-wise Clássica vs. FCN Modela

- **CNN Clássica Por Patch (Sliding Window):** Classificava isoladamente o pixel central de um **patch** deslocado. Isso gerava alta redundância de cálculos e resultava em um efeito de **suavização excessiva nas bordas dos objetos (*oversmoothing*)**.
- **FCN Moderna:** Classifica todos os pixels do **patch** simultaneamente em uma única passada (**dense prediction**), aprendendo estruturas e geometrias específicas diretamente contidas dentro de cada bloco.

Object Detection

2.1 Introdução

Dentro da área de visão computacional, podemos fazer algumas distinções de tarefas, veja o gráfico abaixo

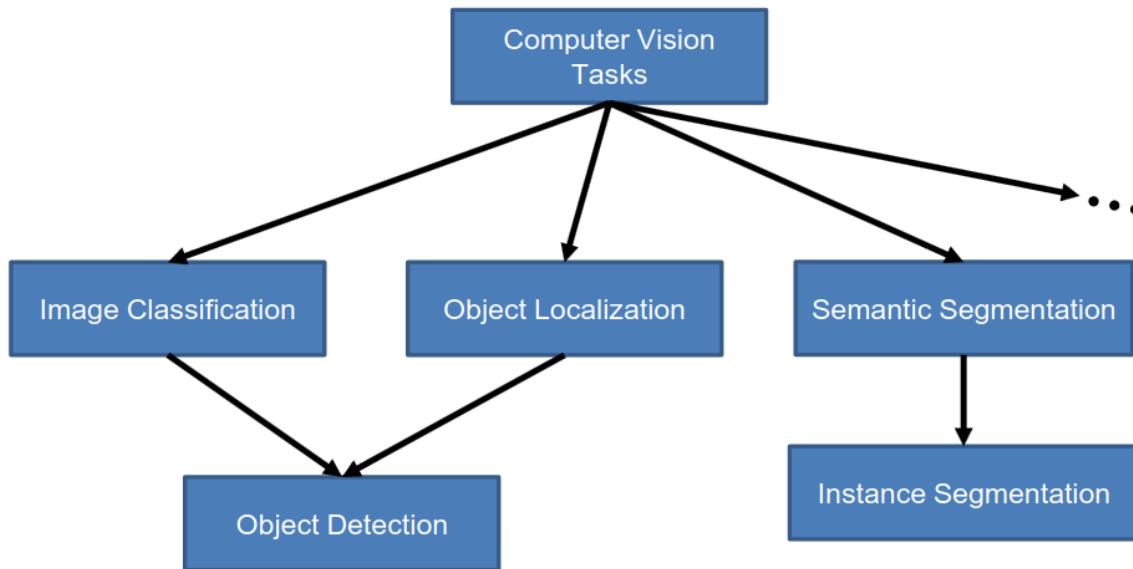


Figura 27: Áreas de visão computacional

Já vimos segmentação semântica, que é a tarefa de classificar cada pixel da imagem em uma categoria específica. A detecção de objetos, por outro lado, é a tarefa de identificar e localizar objetos específicos dentro de uma imagem, geralmente representados por **bounding boxes**. A detecção de objetos é muito baseada em **regiões** enquanto a segmentação semântica é baseada em **pixels**. A detecção de objetos é fundamental para diversas aplicações, como vigilância, direção autônoma e análise de imagens médicas.

2.2 Métricas de Avaliação

Podemos utilizar métricas já vistas como Definição 1.1.1.1, Definição 1.1.1.2 e Definição 1.1.1.3, porém podemos também utilizar métricas como **Recall**

Definição 2.2.1 (Recall):

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (16)$$

A maioria das competições utiliza a **mean Average Precision (mAP)** como métrica principal, que é a média das precisões de cada classe, considerando diferentes limiares de confiança para as detecções. O mAP é derivado de valores *precision v.s recall*, fazendo uma variação do limiar de confiança para cada classe. O **limiar de confiança** é a probabilidade de que uma **caixa de âncora** contenha um objeto. Dado a Definição 1.1.1.3 de Average Precision, podemos definir melhor o mAP

Definição 2.2.2 (mAP): Dado que na minha imagem eu tenho mapeado K classes, a métrica de mean Average Precision (mAP) é definida como a média das precisões de cada classe:

$$\text{mAP} = \frac{1}{K} \sum_{k=1}^K \text{AP}_k \quad (17)$$

2.3 Redes de Estágio Único (Single-Shot): A Família YOLO

Baseada em redes neurais convolucionais que redefiniu o problema de detecção como uma única tarefa de regressão e classificação em uma só passada (**single-shot**).

Ao contrário das abordagens tradicionais baseadas em **sliding window** (janela deslizante), que executavam classificadores repetidamente sobre centenas de retalhos da imagem gerando um custo computacional altíssimo, o YOLO avalia a imagem inteira de uma só vez

2.3.1 Fundamentos do YOLO

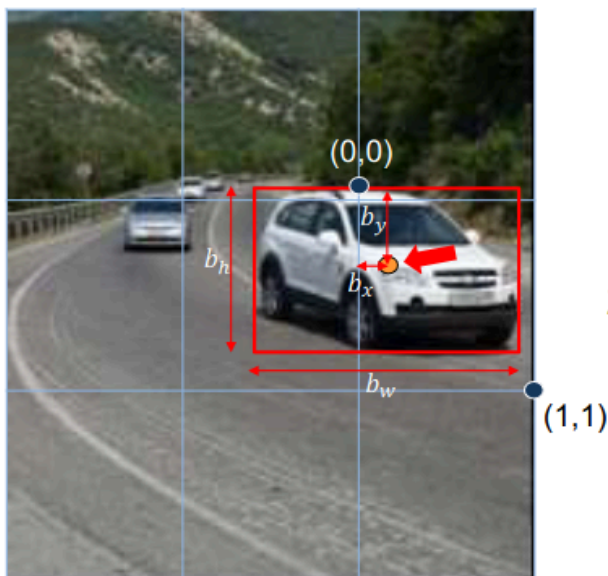
Um modelo YOLO (You Only Look Once) divide a imagem em uma grade de células, ele também recebe obrigatoriamente uma imagem quadrada, de lados $n \times n$. Cada célula da grade fica responsável por prever objetos cujo **ponto central** (mid point) caia dentro dos limites dessa célula. Como uma única passada na CNN processa todas as células simultaneamente, o YOLO é extremamente rápido e eficiente, tornando-o adequado para aplicações em tempo real.

2.3.2 Parametrização do vetor de saída

Para cada célula da grade, o YOLO prevê um vetor de saída que contém informações sobre os objetos detectados. Esse vetor inclui:

$$y = [p_{\text{obj}}, b_x, b_y, b_h, b_w, c_1, c_2, \dots, c_C] \quad (18)$$

- p_{obj} : Probabilidade de que a célula contenha um objeto
- (b_x, b_y, b_h, b_w) : Coordenadas da caixa delimitadora (x, y, largura, altura). $b_x, b_y \in [0, 1]$, representando a posição relativa do centro da caixa em relação à célula. Por exemplo, se $b_x = 0.5$ e $b_y = 0.5$, então a caixa de âncora está exatamente no centro da célula. b_h e b_w representam a altura e largura relativas à caixa, mas podem ser maiores que 1 (a caixa pode ser maior que a célula).



$$y = \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix} \rightarrow \left. \begin{array}{l} \rightarrow 0.2 \\ \rightarrow 0.5 \\ \rightarrow 0.9 \\ \rightarrow 1.5 \end{array} \right\} \begin{array}{l} \text{between 0 and 1} \\ \text{can be } > 1 \end{array}$$

There are many other parametrizations.

Figura 28: Exemplo de caixa delimitadora prevista pelo YOLO

- $(c_1, c_2, \dots, c_C)$: Probabilidades de cada classe

2.3.3 Caixas de Ancoragem (Anchor Boxes)

Para resolver o problema de múltiplos objetos cujos centros caíam na mesma célula ou objetos de proporções muito distintas (como uma pessoa alta e um carro largo), o YOLO utiliza **Anchor Boxes**. Cada célula da grade prevê múltiplos **bounding boxes** associados a modelos geométricos

pré-definidos (**anchors**). Em vez de prever o formato absoluto da caixa do zero, a rede aprende deslocamentos (**offsets**) para ajustar a posição e a dimensão das **anchor boxes pré-definidas**

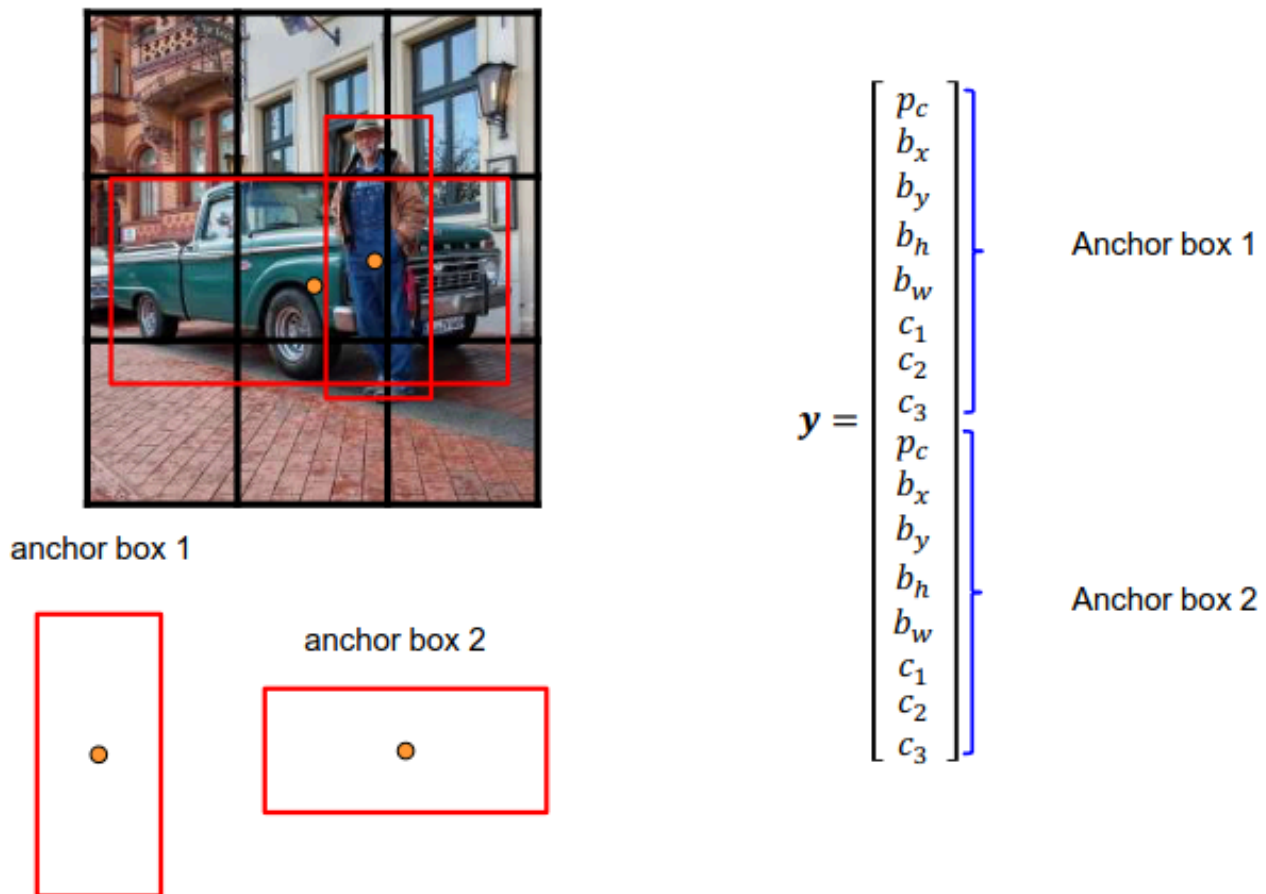


Figura 29: Exemplo de caixas de ancoragem previstas pelo YOLO

Então nessa nova formulação, o vetor de saída para cada célula da grade se torna:

$$y = \begin{pmatrix} [p_{obj}, b_x, b_y, b_h, b_w, c_1, c_2, \dots, c_C]_{\text{anchor 1}} \\ [p_{obj}, b_x, b_y, b_h, b_w, c_1, c_2, \dots, c_C]_{\text{anchor 2}} \\ \vdots \\ [p_{obj}, b_x, b_y, b_h, b_w, c_1, c_2, \dots, c_C]_{\text{anchor A}} \end{pmatrix} \quad (19)$$

Vale ressaltar que eu escrevi em forma de matriz, no entanto o mais comum é um vetor contínuo e separamos as anchor boxes pelo padrão da saída, que seria a cada $5 + C$ valores, onde C é o número de classes. Por exemplo, se temos 3 anchor boxes e 20 classes, o vetor de saída para cada célula da grade terá tamanho $3 \times (5 + 20) = 75$.

2.3.4 Supressão Não-Máxima (Non-Maximum Suppression - NMS)

Na prática, é bem fácil perceber que vai acontecer de várias caixas serem selecionadas para o mesmo objeto, e isso é um problema. Para resolver isso, utilizamos a técnica para escolher a caixa que melhor representa o objeto, descartando as demais. A técnica é chamada de **Non-Maximum Suppression (NMS)**, e funciona da seguinte forma:

1. **Filtragem por confiança:** Descartamos todas as caixas cuja probabilidade de conter um objeto seja menor que um limiar pré-definido (ex: 0.5)
2. **Seleção da melhor caixa:** Entre as caixas restantes, selecionamos a caixa com a maior probabilidade de conter um objeto (maior pontuação de confiança p_{obj})

3. **Eliminação de duplicatas:** Elimina as outras caixas da mesma classe que possuem uma sobreposição $\text{IoU} \geq 0.5$ com a caixa selecionada
4. Repete o processo iterativamente para as caixas restantes de cada classe

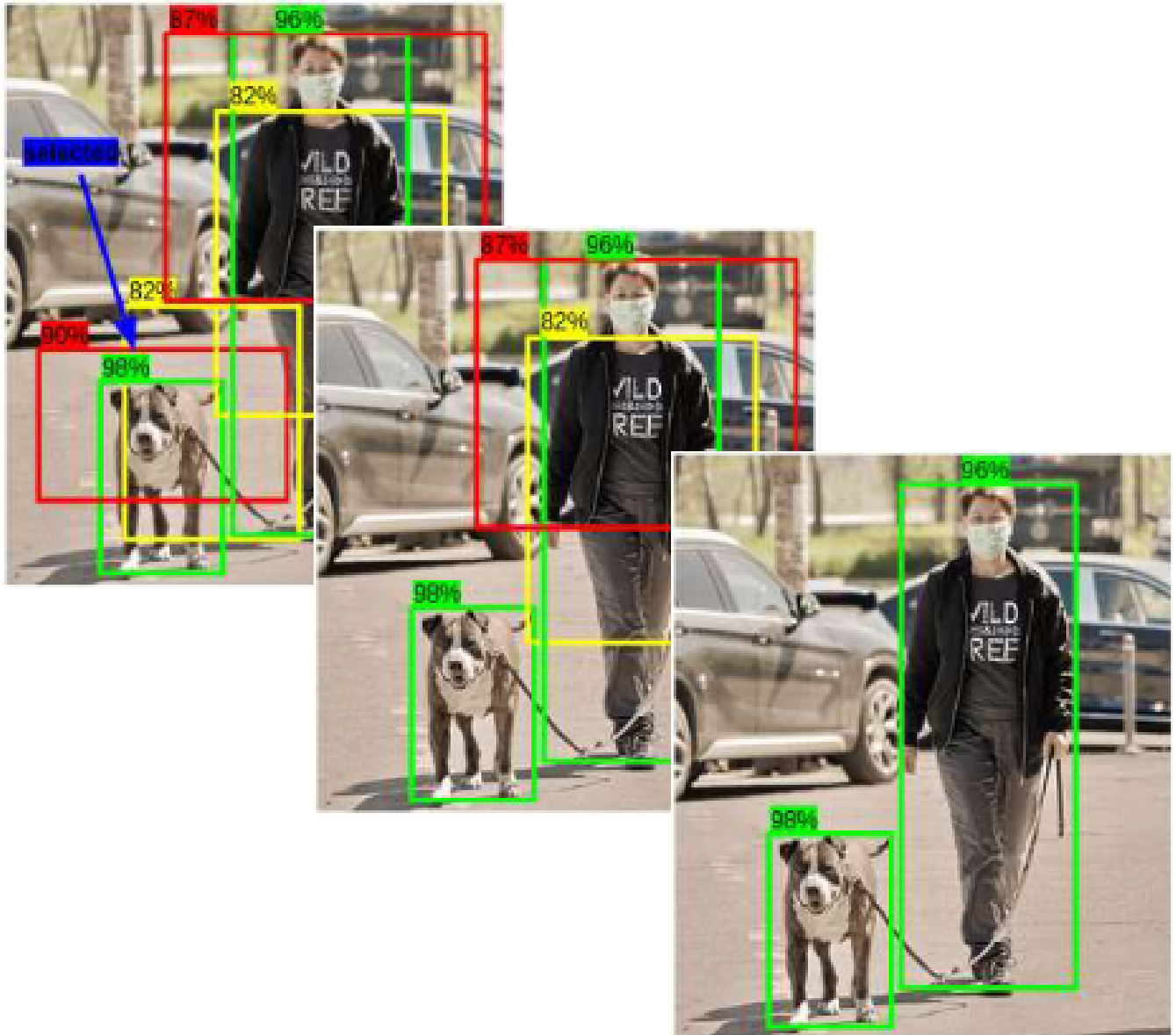


Figura 30: Exemplo de supressão não-máxima

Vale ressaltar que esse **pós-processamento** é feito com **todas** as caixas de ancoragem previstas, tanto as que foram atribuídas dentro de uma mesma célula (uma única célula atribui diferentes anchor boxes pro mesmo objeto) quanto as geradas por células vizinhas (várias células podem prever o mesmo objeto). O objetivo é garantir que cada objeto seja representado por uma única caixa delimitadora final.

2.3.5 Evolução Arquitetural

2.3.5.1 YOLOv1 & YOLO9000

Arquiteturas iniciais, foi na YOLO9000 onde as anchor boxes foram introduzidas e, em vez de prever diretamente posição e tamanho das caixas, a rede aprende a prever **offsets** para ajustar as **anchor boxes** pré-definidas.

2.3.5.2 YOLOv3

Aumento da profundidade da rede, de 53 camadas para 106 camadas. Adição de skip connections para melhorar a propagação do gradiente e permitir que a rede aprenda representações mais complexas. Introdução de **multi-scale predictions**, onde a rede prevê caixas em três escalas diferentes, permitindo detectar objetos de tamanhos variados.

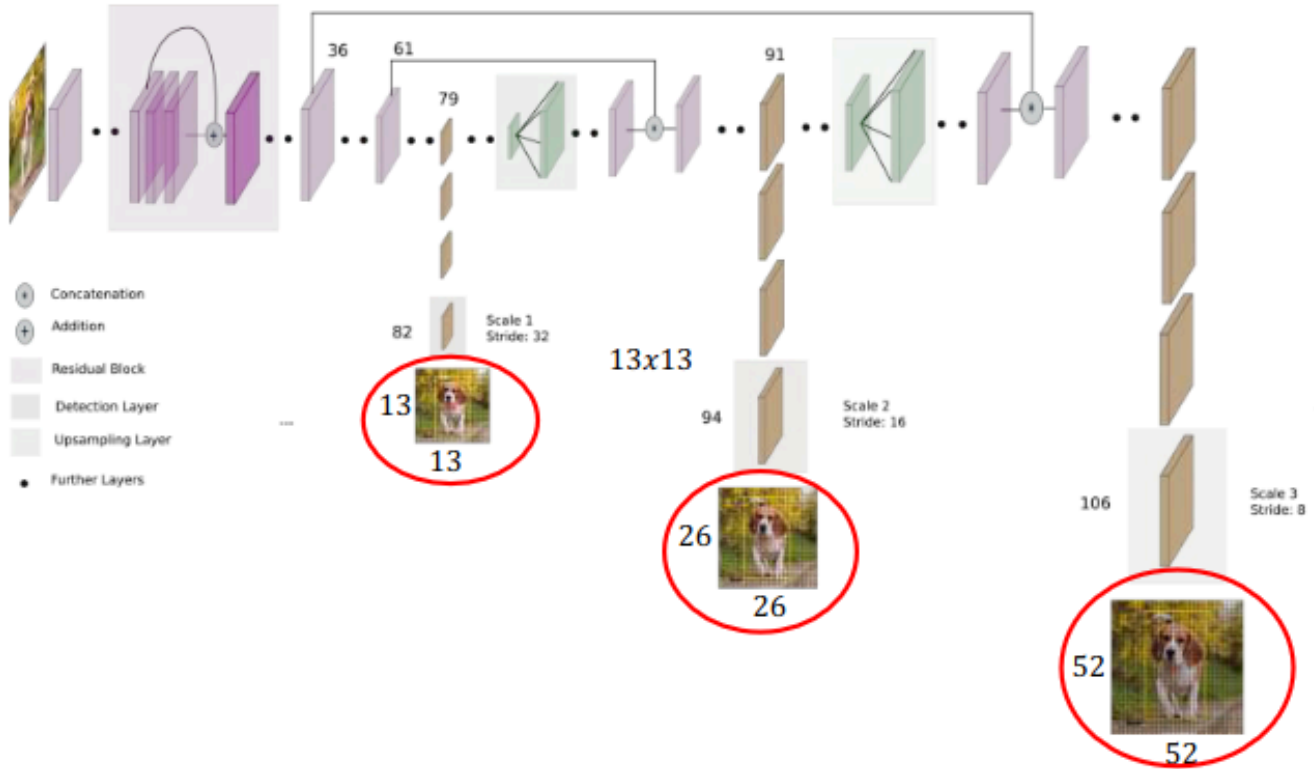


Figura 31: Arquitetura da YOLOv3

2.3.6 Loss Function

Não podemos falar de um modelo de rede sem discutir a loss por ela utilizada. A loss do YOLO é composta por três partes principais: a **loss de localização**, a **loss de confiança** e a **loss de classificação**. A **loss de localização** mede o quão bem a rede prevê as coordenadas da caixa delimitadora em relação à caixa real. A **loss de confiança** avalia a precisão da rede em prever se uma caixa contém um objeto ou não. A **loss de classificação** mede a precisão da rede em classificar corretamente o objeto dentro da caixa.

Vamos rapidamente definir que a **predição do modelo** é dada, para a célula $i \in \{1, \dots, S^2\}$ e anchor box $j \in \{1, \dots, B\}$:

$$y_{ij} = [p_{\text{obj}}^{(ij)}, b_x^{(ij)}, b_y^{(ij)}, b_h^{(ij)}, b_w^{(ij)}, c_1^{(ij)}, c_2^{(ij)}, \dots, c_C^{(ij)}] \quad (20)$$

e vamos considerar o vetor **ground truth** como

$$t_{ij} = [t_0^{(ij)}, t_x^{(ij)}, t_y^{(ij)}, t_h^{(ij)}, t_w^{(ij)}, s_1^{(ij)}, s_2^{(ij)}, \dots, s_C^{(ij)}] \quad (21)$$

então a loss da YOLO é dada por

$$\begin{aligned}
\mathcal{L}_{\text{YOLO}} = & \underbrace{\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B t_0^{(ij)} \left(\left(t_x^{(ij)} - b_x^{(ij)} \right)^2 + \left(t_y^{(ij)} - b_y^{(ij)} \right)^2 + \left(t_h^{(ij)} - b_h^{(ij)} \right)^2 + \left(t_w^{(ij)} - b_w^{(ij)} \right)^2 \right)}_{\text{Loss de localização}} \\
& + \underbrace{\lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \left(1 - t_0^{(ij)} \right) \left(-\log \left(1 - p_{\text{obj}}^{(ij)} \right) \right)}_{\text{Loss de confiança}} \\
& + \underbrace{\sum_{i=0}^{S^2} \sum_{j=0}^B t_0^{(ij)} \left[-\log \left(p_{\text{obj}}^{(ij)} \right) + \sum_{k=1}^C \text{BCE} \left(c_k^{(ij)}, s_k^{(ij)} \right) \right]}_{\text{Loss de classificação}}
\end{aligned} \tag{22}$$

Onde S é a proporção que dividimos os grids da imagem, B é o número de **anchor boxes** por célula, C é o número de classes, λ_{noobj} e λ_{coord} são hiperparâmetros que controlam a importância relativa das diferentes partes da loss.

Vejamos como a loss se comporta. Se uma anchor box **não possui objeto**, então apenas a parte da **loss de confiança** é computada. Quando a probabilidade de que a caixa contenha um objeto é 0, então a $\log(1 - 0) = 0$, não penalizando a rede, no entanto, se a rede classificou como existindo um objeto, então a loss será alta e irá penalizar a rede. Se uma anchor box **possui objeto**, então apenas as partes da **Loss de Localização** e **Loss de Classificação** são computadas. A **Loss de Localização** mede o quão bem a rede prevê as coordenadas da caixa delimitadora em relação à caixa real, e a **Loss de Classificação** mede a precisão da rede em classificar corretamente o objeto dentro da caixa.

2.4 Redes de Dois Estágios e Segmentação de Instâncias: Mask R-CNN

Recurrent Neural Networks (RNNs)

Generative Adversarial Networks (GANs)

